Práctica 2

Control del flujo y E/S en CLIPS

Sistemas Basados en Reglas (SBR)

1 Introducción

En la práctica anterior, el explorador recorrió la Meseta del Alba y alcanzó el Pico helado, donde consiguió entrar en calor junto a la hoguera. Desde allí, otea el horizonte y descubre al este una atalaya en ruinas. En ella, un guardián, un autómata antiguo cubierto de escarcha, custodia una paravela: el único medio de abandonar una meseta rodeada de acantilados.

El objetivo de esta práctica es que el explorador consiga la paravela y salte desde el pico para planear fuera de la meseta. Por el camino, el jugador dejará de utilizar `assert` y escribirá directamente sus órdenes. Para ello se verán el elemento condicional `not`, las operaciones de entrada y salida, las estructuras condicionales e iterativas y los atributos `multislot`. Al final del guion, una partida tendrá un aspecto similar al siguiente:

```
> ir este
[Atalaya en ruinas] Una torre derruida al borde del acantilado.
Ves: paravela
El ojo del guardian se vuelve rojo y fija su mirada en ti.
> esconderse
Te ocultas tras una columna caida.
El guardian gira la cabeza, buscandote entre las ruinas.
> usar tablilla
Apoyas la tablilla en el pecho del guardian. El ojo del guardian se ha apagado.
```

1.1 Punto de partida

La práctica continúa sobre el mismo `Meseta.clp` de la práctica anterior. Para garantizar que el punto de partida es idéntico, independientemente de si se finalizó o no la práctica anterior, se facilita en el propio zip el fichero `Meseta.clp` que contiene, además del mínimo necesario para garantizar que este guión puede ejecutarse sin problemas, una serie de reglas adicionales que le permiten comenzar desde el Pico en lugar de la Camara.

Junto con el guion se adjunta también `Codigo_P2.txt`, que reúne todos los bloques de código con el mismo número que aparece junto a cada uno; utilícelo si el código no se pega correctamente al copiarlo desde el PDF.

2 La atalaya y su guardián

El primer paso es llevar al explorador hasta la atalaya. Añada a `Meseta.clp` los siguientes hechos iniciales, que describen la atalaya y la paravela:

1

```
(deffacts atalaya
  (lugar (id atalaya) (nombre "Atalaya en ruinas")
    (descripcion "Una torre derruida al borde del acantilado."))
  (camino pico este atalaya abierto)
  (camino atalaya oeste pico abierto)
  (entidad (id paravela) (categoria objeto) (lugar atalaya)
    (descripcion "Una tela resistente tensada que permite planear.")))
```

En este guión pasará a tomar el control sobre partes de la implementación, empezando por el diseño del guardián. Este autómata tiene un identificador, el lugar que vigila y una descripción, que se mostrará al examinarlo. Además, siempre se encuentra en uno de estos estados: `patrulla`, mientras recorre el lugar; `alerta`, cuando detecta al explorador; `despistado`, cuando este consigue esconderse; y `desactivado`, cuando el explorador le apoya la tablilla en el pecho. Al comienzo de la partida está en `patrulla`.

Ejercicio 1

1. Diseñe la plantilla `guardian` a partir de la descripción anterior.
2. Añada a los hechos iniciales de la atalaya un guardián que la vigile, con la descripción «Un automata antiguo cubierto de escarcha.». ¿Qué ocurre si intenta insertar un guardián con un estado que no está permitido? ¿Qué estado toma si no se indica?
3. El estado del guardián también podría representarse con un hecho no estructurado, como `(estado guardian patrulla)`, del mismo modo que la resistencia. ¿Qué ventajas tiene la plantilla?

3 El elemento condicional `not`

Con los hechos anteriores, `recoger` permitiría al explorador llevarse la paravela delante del guardián, que debería impedirlo mientras siga activo. Hasta ahora, todas las condiciones de una regla eran patrones que debían encajar con algún hecho. El elemento condicional `(not <patron>)` hace lo contrario: se satisface si no existe ningún hecho que encaje con el patrón. Mientras que un patrón normal genera una activación por cada hecho que encaja, `not` genera, como mucho, una.

La siguiente regla intercepta la orden antes que `recoger`, gracias a su mayor prioridad, mientras el guardián no esté desactivado:

```
(defrule paravela_custodiada
  (declare (salience 10))
  ?o <- (orden recoger | tomar $? paravela)
  (entidad (id explorador) (lugar atalaya))
  ; Incluya aqui el patron del guardian segun su propia implementacion
  =>
  (retract ?o)
  (println "No parece seguro coger la paravela en este momento."))
```

En este caso, la condición también podría expresarse con la restricción `~` de la práctica anterior, comprobando que el estado del guardián es `~desactivado`. La diferencia aparece cuando no existe el hecho: si no hubiera ningún guardián, el patrón con `~` no encajaría con ningún hecho, mientras que `not` seguiría satisfaciéndose.

Las variables que aparecen dentro de `not` solo sirven para relacionar campos dentro del propio patrón, ya que no se refieren a ningún hecho concreto, y no pueden utilizarse en el consecuente.

Ejercicio 2

1. Añada `paravela_custodiada`, completándola con un patrón de su guardián que utilice `not` para comprobar que no está desactivado. Cree después la regla `detectar`: cuando el explorador llegue al lugar que vigila un guardián en `patrulla`, este pasa a `alerta` y se muestra «El ojo del guardian se vuelve rojo y fija su mirada en ti.». ¿Por qué conviene relacionar mediante una variable el lugar del explorador con el del guardián, en lugar de escribir directamente `atalaya`?
2. Desde el pico, la orden `(orden ir norte)` muestra «No entiendes muy bien lo que quieres hacer.», ya que no hay ningún camino en esa dirección y la orden termina retirándola `orden_desconocida`. Cree la regla `sin_camino`, que muestre «No hay ningun camino hacia el norte.» (o la dirección indicada) cuando no exista ningún camino, abierto o cerrado, desde el lugar del explorador en esa dirección.

Tras el flujo habitual, inserte `(orden ir este)` y `(orden recoger paravela)`, ejecutando `(run)` tras cada una: el guardián debe ponerse en alerta e interponerse, y la paravela debe seguir en la atalaya.

4 Lectura de órdenes: `read` y `readline`

Introducir cada orden con `assert` y ejecutar `(run)` tras ella puede llegar a resultar tedioso si la simulación es amplia, por lo que el siguiente paso es que el jugador pueda escribir sus órdenes directamente. Para ello, CLIPS dispone de dos instrucciones de lectura: `(read [<nombre-logico>])` y `(readline [<nombre-logico>])`. El nombre lógico es opcional e indica desde dónde se lee: un fichero o, si se omite o se escribe `t`, el teclado. `read` devuelve un único campo y se detiene en el primer espacio, tabulador o salto de línea, salvo que el texto esté entre comillas dobles. `readline`, en cambio, devuelve la línea completa como cadena.

En el caso de las órdenes interesa leer la línea completa y convertirla en una lista de palabras. Para ello se utilizarán otras dos funciones: `lowcase`, que pasa el texto a minúsculas, y `explode$`, que convierte una cadena en varios campos. Así, si el jugador escribe `Recoger Paravela`, la regla insertará el hecho `(orden recoger paravela)`:

3
```
(defrule pedir_orden
  (declare (salience -100))
  (not (orden $?))
  (not (fin))
  =>
  (print "> ")
  (assert (orden (explode$ (lowcase (readline))))))
```

Al tener la prioridad más baja del sistema, la regla solo se ejecuta cuando no queda nada en la agenda, es decir, cuando ya se ha procesado la orden anterior y no existe ninguna otra. La condición `(not (fin))` permite, además, terminar la partida en cualquier momento creando el hecho `(fin)`. ¿Por qué no impide la refracción que la regla vuelva a ejecutarse tras cada orden?

Algunos terminales, como el de Visual Studio Code, terminan cada línea con un carácter adicional que `readline` lee como una línea vacía, de modo que tras cada orden aparecería el aviso de `orden_desconocida`. La siguiente regla descarta esas órdenes vacías antes de que lo haga `orden_desconocida`:

4
```
(defrule orden_vacia
  (declare (salience -5))
  ?o <- (orden)
  =>
  (retract ?o))
```

Ejercicio 3

Añada `pedir_orden` y `orden_vacia`, y cree una regla para la orden `salir` que inserte el hecho `(fin)` y se despida del jugador. A partir de ahora, tras `(reset)` y `(run)`, la partida se juega escribiendo las órdenes directamente, sin paréntesis. Compruebe que `ir este` lleva al explorador a la atalaya, que `recoger paravela` sigue sin funcionar y que con la orden `salir` se devuelve el control al intérprete.

5 Estructuras condicionales: `if` y `switch`

CLIPS incluye estructuras selectivas, `if` y `switch`, que técnicamente pueden escribirse tanto en funciones como en el consecuente de una regla. Sin embargo, en la asignatura sólo se permitira su uso en funciones. Esto se debe a que una decisión dentro del consecuente queda oculta al motor de inferencia: la condición no aparece en el antecedente, no se refleja en la agenda y la regla se activa aunque después no haga nada. En un SBR, cada caso debe expresarse como una regla distinta, con su condición en el antecedente, y es el motor de inferencia quien decide cuál aplicar.

Por ejemplo, la siguiente versión de `abrir_puerta`, de la práctica anterior, es incorrecta: se activa esté donde esté la tablilla y decide en su consecuente si debe actuar.

```
(defrule abrir_puerta_incorrecto          ; NO: if en el consecuente
  (entidad (id tablilla) (lugar ?l))
  ?c <- (camino camara norte explanada cerrado)
  =>
  (if (eq ?l explorador)
    then (retract ?c)
         (assert (camino camara norte explanada abierto))
         (println "La tablilla se ilumina y la puerta se abre con un crujido.")))
```

La versión original expresa esa misma condición mediante el patrón `(entidad (id tablilla) (lugar explorador))`, de forma que solo se activa cuando debe actuar. Si se necesitara la rama `else`, se escribiría otra regla con la condición contraria, como `paravela_custodiada` y `recoger`.

5.1 `if`

La sintaxis de la estructura condicional es `(if <expresion> then <accion>* [else <accion>*])`. Al igual que una función, devuelve el valor de la última acción ejecutada, por lo que, dentro de una función, permite devolver un resultado u otro, como en `(if (> ?r 70) then "Con fuerzas" else "Cansado")`.

5.2 `switch`

También es posible realizar una selectiva múltiple:

5

```
(switch <expresion-test>
  (case <valor-1> then <accion>*)
  (case <valor-2> then <accion>*)
  ...
  [(default <accion>*)])
```

Veamos un pequeño ejemplo. La siguiente función describe lo que hace el guardián en cada uno de sus estados:

6

```
(deffunction actitud (?estado)
  (switch ?estado
    (case patrulla then "El guardian recorre la atalaya con paso lento.")
    (case alerta then "El guardian fija en ti su ojo rojo.")
    (case despistado then "El guardian gira la cabeza, buscandote entre las ruinas.")
    (case desactivado then "El ojo del guardian se ha apagado.")
    (default "El guardian permanece inmovil.")))
```

Ejercicio 4

1. Añada la variable global `?*coste-planear*` con valor 25, que será la resistencia necesaria para planear desde el pico. Cree una función `estado_actual` que, a partir de la resistencia del explorador, devuelva un texto distinto según esté por encima de 70, por encima o igual al coste de planear o por debajo de este. Utilice `if` anidados. Tras esto, cree una regla que, cuando reciba la orden `estado`, muestre la resistencia del explorador junto con el texto que devuelve la función.
2. Añada la función `actitud` a `Meseta.clp` y cree la regla `esconderse`, que atiende la orden del mismo nombre: si el explorador se encuentra en el mismo lugar que un guardián en `alerta`, este pasa a `despistado` y se muestra «Te ocultas tras una columna caida.». Finalmente, la regla deberá imprimir la actitud del guardián en su nuevo estado llamando a la función `actitud`.

Al comienzo, `estado` debe mostrar resistencia 60 y su texto. En la atalaya, tras `esconderse`, debe mostrarse que el guardián está despistado.

6 Atributos multivalor: `multislot`

Hasta ahora, cada atributo de una plantilla contenía un solo valor mediante `slot`. Un `multislot` permite almacenar cero o más valores en el mismo atributo. Así, la tablilla puede llevar un mapa con la lista de lugares que ha visitado el explorador, que al comenzar la partida solo contiene el pico:

```
(deftemplate mapa
  (multislot visitados))

(deffacts mapa_inicial
  (mapa (visitados pico)))
```

La siguiente regla anota en el mapa el lugar en el que se encuentra el explorador, siempre que todavía no figure en él. El patrón `(visitados $? ?l $?)` busca `?l` en cualquier posición de la lista, mientras que `$?v` conserva la lista completa para añadir el nuevo lugar al final:

```
(defrule anotar_lugar
  (entidad (id explorador) (lugar ?l))
  ?m <- (mapa (visitados $?v))
  (not (mapa (visitados $? ?l $?)))
  =>
  (modify ?m (visitados $?v ?l))
  (println "La tablilla anota en el mapa: " ?l))
```

Observe que `modify` sustituye la lista completa por la nueva. Del mismo modo, para eliminar un valor basta con dar nombre a lo que hay antes y después de él: con el patrón `(visitados $?antes templo $?despues)`, `(modify ?m (visitados $?antes $?despues))` quitaría el templo del mapa. Fíjese también en que `multislot` describe cómo se almacenan los datos, mientras que `$?` permite buscar o conservar varios campos en un patrón.

7 Operaciones con ficheros y `while`

Para llevarse la paravela, el explorador debe desactivar al guardián apoyando la tablilla en su pecho mientras está despistado. Al hacerlo, el guardián reproduce un mensaje que lleva grabado desde hace siglos y que se almacenará en el fichero `memoria_guardian.txt`.

Para leer o escribir en un fichero es necesario abrirlo previamente, asociándole un nombre lógico, mediante `(open <fichero> <nombre-logico> [<modo>])`, donde el modo puede ser `"r"` (lectura, por defecto), `"w"` (escritura) o `"a"` (añadir al final). A partir de ese momento, el nombre lógico puede utilizarse con `read`, `readline` o `printout`, y el fichero se cierra al terminar con `(close [<nombre-logico>])`. Veamos un ejemplo en el intérprete:

```
CLIPS> (open "datos.txt" misdatos "w")
TRUE
CLIPS> (printout misdatos "rojo verde" crlf)
CLIPS> (close misdatos)
TRUE
CLIPS> (open "datos.txt" misdatos)
TRUE
CLIPS> (readline misdatos)
"rojo verde"
CLIPS> (readline misdatos)
EOF
CLIPS> (close misdatos)
TRUE
```

Tenga en cuenta que un nombre de fichero sin ruta, como `"datos.txt"`, se busca en el directorio de trabajo de CLIPS. Por eso es importante que antes de trabajar con ficheros lo establezca con `(chdir "<carpeta de la practica>")`, escribiendo la ruta completa de la carpeta que contiene `Meseta.clp`.

Como no se sabe de antemano cuántas líneas tiene el mensaje, el fichero se leerá con un bucle `(while <expresion> [do] <accion>*)`, que repite sus acciones mientras se cumpla la condición. La siguiente función muestra el fichero línea a línea hasta encontrar `EOF`:

9

```
(deffunction reproducir_memoria ()
  (if (open "memoria_guardian.txt" memoria "r")
    then (bind ?linea (readline memoria))
         (while (neq ?linea EOF) do
           (println ?linea)
           (bind ?linea (readline memoria)))
         (close memoria)
    else (println "El guardian emite un zumbido ininteligible.")))
```

`open` devuelve `FALSE` si el fichero no se puede abrir, por lo que el `if` evita leer de un fichero que no existe.

Ejercicio 5

1. Cree con un editor de texto el fichero `memoria_guardian.txt`, en la carpeta de la práctica, con estas tres líneas:

   10

   ```
   Unidad de vigilancia de la atalaya. Ultima orden recibida hace cien anos.
   Custodiar la paravela hasta que regrese el portador de la tablilla.
   Portador reconocido. Mision completada.
   ```

2. Añada la función `reproducir_memoria`. Después, cree la regla `desactivar_guardian`, que atiende las órdenes que empiezan por `usar` y terminan en `tablilla`, como `usar tablilla`. La regla solo debe activarse si el explorador lleva la tablilla y se encuentra en el mismo lugar que un guardián `despistado`. En ese caso, el guardián pasa a `desactivado`, se muestra «Apoyas la tablilla en el pecho del guardian.» seguido de su actitud y se llama a `reproducir_memoria`.
3. Compruébelo en la atalaya: escóndase, use la tablilla y recoja la paravela. Tras usar la tablilla deben aparecer las tres líneas del mensaje, y `recoger paravela` debe funcionar. ¿Qué ocurre si intenta usar la tablilla mientras el guardián está en alerta? ¿Y si cambia el nombre del fichero?

8 Estructuras iterativas: `loop-for-count`

Con la paravela, el explorador ya dispone de lo necesario para abandonar la meseta: saltar desde el pico y planear. Cuando se conoce de antemano el número de iteraciones, puede utilizarse `(loop-for-count (<variable-indice> <inicio> <final>) [do] <accion>*)`, el equivalente a un bucle `for` de otros lenguajes de programación. La variable índice puede omitirse cuando no se necesita, como en `(loop-for-count 3 (println "zzz"))`.

El descenso consta de cinco tramos, y en cada uno el explorador pierde 5 puntos de resistencia. El bucle se escribe en una función, `planear`, que recibe la resistencia inicial y devuelve la que le queda al explorador al tocar tierra:

11
```
(deffunction planear (?r)
  (loop-for-count (?tramo 1 5) do
    (bind ?r (- ?r 5))
    (println "Planeas... altura " (- 5 ?tramo) ", resistencia " ?r))
  ?r)    ; devuelve la resistencia al tocar tierra
```

La regla `saltar` llama a la función en su consecuente y utiliza el valor que devuelve para crear el nuevo hecho de resistencia. Al terminar, el explorador queda en un nuevo lugar, `llanura`, fuera de la meseta:

12
```
(defrule saltar
  ?o <- (orden saltar)
  ?e <- (entidad (id explorador) (lugar pico))
  (entidad (id paravela) (lugar explorador))
  ?f <- (resistencia ?r)
  (test (>= ?r ?*coste-planear*))
  =>
  (retract ?o ?f)
  (println "Saltas desde el pico y despliegas la paravela.")
  (assert (resistencia (planear ?r)) (fin))
  (modify ?e (lugar llanura))
  (println "Tocas tierra fuera de la meseta. La Gran Llanura se extiende ante ti."))
```

Observe que la función recibe una copia del valor de `?r`: el `bind` dentro de `planear` no modifica el hecho `resistencia`. Por eso la regla retira el hecho original y crea otro con el valor que devuelve la función.

Añada `planear` y `saltar`, y juegue una partida completa desde el pico. Con esto, el juego puede jugarse de principio a fin: el explorador abandona la meseta y le espera la Gran Llanura.

9 Entregas

El objetivo de la práctica es que se completen los ejercicios ampliando el fichero `Meseta.clp` y, si le da tiempo comience a implementar el siguiente ejercicio en un fichero adicional `Mercader.clp`. En todo caso se le recuerda que debe entregar hasta donde le haya dado tiempo, entregando sólo la ampliación de `Meseta.clp` o bien un fichero zip con ambos ficheros.

9.1 Los mercaderes de la posta (Mercader.clp)

Al pie de la meseta hay una posta en la que se detienen los mercaderes ambulantes. Esta tarea no depende de `Meseta.clp` por lo que deberá implementar las plantillas, las variables globales, las funciones y las reglas de un SBR que gestione las compras y ventas del explorador a partir de la siguiente descripción.

mercader identificador, nombre (cadena) y lugar en el que se encuentra.

producto identificador, tipo (`comida`, `material` o `equipo`) y valor base en rupias (entero).

oferta relaciona un mercader con un producto que vende, con el descuento que hace al cliente sobre el valor base (entero, 0 por defecto) y sus existencias (entero, 1 por defecto). Un mismo producto puede venderlo varios mercaderes, cada uno con su propio descuento.

cliente identificador, lugar y rupias (entero, 0 por defecto).

inventario relaciona un cliente con un producto que lleva y su cantidad (entero, 1 por defecto).

peticion cliente, acción (`comprar` o `vender`) y producto.

Los precios de venta incluyen un impuesto según el tipo de producto, cuyos valores se representarán con variables globales: un 10 % para los materiales y un 20 % para el equipo.

Con sus plantillas, cree los hechos iniciales a partir de los siguientes datos de prueba. Hay dos mercaderes: `m1`, el «Mercader ambulante», que se encuentra en la `posta`, y `m2`, la «Herrera del lago», en el `lago`. Los productos, con su tipo y su valor base, son la `manzana` (comida, 3), la `flecha` (equipo, 2), el `escudo` (equipo, 40) y el `ambar` (material, 30). El único cliente es el `explorador`, que está en la `posta` con 30 rupias y lleva un `ambar`. Por último, las ofertas de cada mercader se recogen en la Tabla 1; los valores que coincidan con el valor por defecto de su plantilla pueden omitirse.

Mercader	Producto	Descuento	Existencias
`m1`	`manzana`	0	10
`m1`	`flecha`	0	20
`m1`	`escudo`	10	0
`m1`	`ambar`	5	3
`m2`	`escudo`	5	1
`m2`	`flecha`	0	5

Tabla 1: Ofertas iniciales de los mercaderes

Tareas

1. Defina las plantillas, las variables globales y los hechos iniciales.
2. Si el cliente está en el mismo lugar que un mercader que ofrece el producto, quedan existencias y le alcanzan las rupias, el producto pasa a su inventario (si ya lo llevaba, aumenta su cantidad; si no, se añade con cantidad 1; para distinguir ambos casos puede usar dos reglas de compra o un hecho auxiliar), se descuentan las rupias y las existencias, y se muestra un mensaje con todos los datos, como «explorador compra flecha a Mercader ambulante por 2 rupias (pedia 2). Le quedan 28.». El precio se calcula mediante una función: valor base menos el descuento, al que se suma el impuesto que corresponda según el tipo de producto (la comida no tiene impuesto).
3. Si al cliente no le alcanzan las rupias, el mercader regatea: rebaja el precio de rupia en rupia hasta que el cliente pueda pagarlo, pero como mucho renuncia al impuesto, es decir, nunca baja del valor base con el descuento aplicado. Calcule el precio final con una función que utilice `while` y úsela en la regla de compra. Recuerde que, en el antecedente, `test` también puede comprobar el resultado de funciones propias.
4. Muestre un aviso, cada uno en su propia regla, cuando no haya ningún mercader en el lugar del cliente, el mercader no tenga el producto (porque no lo ofrece o porque se le ha agotado) o el cliente no pueda pagar ni siquiera tras regatear. Decida mediante prioridades qué aviso aparece si se dan varios a la vez.
5. El cliente puede vender cualquier producto de su inventario al mercader de su lugar, que le paga su valor base. La cantidad del producto se reduce en una unidad y, si llega a 0, el producto desaparece del inventario. Si el cliente no lleva el producto, se le avisa.
6. Haga una prueba completa del sistema, comprobando que todas las reglas y funciones cumplen con lo esperado.